

2.1. Considerando el esquema de base de datos Chinook. Se pide:

- (a) Mediante SQL, AR y CRT listar Nombres (*FirstName* y *LastName*) de los clientes (*Customer*) de Brasil.

a) select first-name, last-name
from Customer
where Country = "Brasil";

SQL

AR

$\rho(\text{Res}, \pi_{\text{FN, LN}} (\sigma_{\text{Country} = \text{"Brasil"}} (\text{Customer})))$

CRT

$\{t \mid \exists c \quad c \in \text{Customer} \wedge c.\text{Country} = \text{"Brasil"} \wedge t.\text{fn} = c.\text{fn} \wedge t.\text{ln} = c.\text{ln}\}$

(b) Mediante SQL, AR y CRT listar para cada cliente (*Customer*) las facturas (*Invoice*) que tiene. Se deberá mostrar el nombre del cliente, la fecha y número de factura (*InvoiceDate* e *InvoiceID*).

SQL:

```

Select  C. FirstName,
        C. LastName,
        I. InvoiceID,
        I. InvoiceDate
From    Customer C
Inner Join Invoice I
        ON I.CustomerID = C.CustomerID
    
```

AR:

$\rho(I, \text{Invoice})$

$\rho(C, \text{Customer})$

$\rho(\text{Cust-Con-Inv}, \text{Customer} \times \text{Invoice})$
 $\quad \quad \quad \text{CCI} \quad \quad \quad \begin{matrix} \text{C.CustomerID} \\ = \\ \text{I.CustomerID} \end{matrix}$

$\rho(\text{Res}, \pi_{\text{CCI.Fn}, \text{CCI.Ln}, \text{CCI.InvoiceDate}, \text{CCI.InvoiceID}}(\text{CCI}))$

CRT: $\{ t / \exists I, C \left(\begin{array}{l} I \in \text{Invoice} \wedge \\ C \in \text{Customer} \wedge \\ I.\text{CustID} = C.\text{CustID} \wedge \\ t.\text{Fn} = C.\text{Fn} \wedge t.\text{Ln} = C.\text{Ln} \wedge \\ t.\text{InvID} = I.\text{InvID} \wedge \\ t.\text{InvDate} = I.\text{InvDate} \end{array} \right) \}$

(c) Mediante SQL, AR y CRT listar, para cada *track*, el nombre del artista (*Artist*) que hizo el *Album* al que pertenece dicho *track*.

SQL:

```
select t.Name, a.Name, alb.title
from track t
inner join album alb
on t.albumId = alb.albumId
inner join Artist a
on alb.ArtistId = a.artistId;
```

AR:

$\rho(t, \text{track})$

$\rho(\text{Alb}, \text{Album})$

$\rho(A, \text{Artist})$

$\rho(\text{Res}, \pi_{t.\text{name}, A.\text{name}, \text{Alb.title}} \left(\left(A \bowtie_{\begin{matrix} A.\text{artistId} \\ = \\ \text{Alb.artistId} \end{matrix}} \text{Alb} \right) \bowtie_{\begin{matrix} \text{Alb.AlbId} \\ = \\ t.\text{AlbId} \end{matrix}} T \right)$

CRT:

$\left\{ \begin{array}{l} t / \exists t, alb, art \quad \begin{array}{l} t \in \text{Track} \wedge \\ alb \in \text{Album} \wedge \\ art \in \text{Artist} \wedge \\ alb.\text{artistId} = art.\text{artistId} \wedge \\ alb.\text{AlbId} = t.\text{AlbId} \wedge \\ t.\text{ nomb-track} = t.\text{name} \wedge \\ t.\text{ nomb-art} = art.\text{name} \wedge \\ t.\text{ Alb} = alb.\text{title} \end{array} \end{array} \right\}$

(d) Mediante AR y CRT obtener los nombres de las *Playlist* que tienen más de un *track* cuyo *MediaType* es "MPEG audio file"

AR

$\rho(P, Playlist)$

$\rho(PLT, Playlist + Track)$

$\rho(T, Track)$

$\rho(M, MediaType)$

$\rho(MPEG, \sigma_{Name = "MPEG..."}(M))$

$\rho\left(\begin{array}{c} TMPEG \\ " \\ Tracks \\ MPEG \end{array}, \left(\begin{array}{c} MPEG \quad (X) \quad T \\ MPEG.MediaTypeId \\ = \\ T.MediaTypeId \end{array}\right)\right)$

$\rho\left(\begin{array}{c} PLT_MPEG \end{array}, \left(\begin{array}{c} TMPEG \quad (X) \quad PLT \\ TMPEG.TrackId \\ = \\ PLT.TrackId \end{array}\right)\right)$

$\rho(Res, \pi_{P.name} \left(\begin{array}{c} PH.MPEG \quad (X) \quad P \\ PLT.MPEG.PlaylistId \\ = \\ P.PlaylistId \end{array}\right))$

CRT:

$$\left\{ t \mid \exists tc, pl, plt, n \left(\begin{array}{l} tc \in \text{Track} \wedge pl \in \text{Playlist} \\ \wedge plt \in \text{PlaylistTrack} \wedge \\ n \in \text{MediaType} \\ \text{MPeg}(n) \wedge tc.\text{MediaType} = \\ \quad n.\text{MediaType} \\ \wedge tc.\text{TrackId} = plt.\text{TrackId} \\ \wedge plt.\text{PlaylistId} = pl.\text{PlaylistId} \\ \wedge t.\text{Nombre} = pl.\text{name} \end{array} \right) \right\}$$

$$\text{MPeg}(n) = n.\text{name} = \text{"MPeg..."}$$

- (e) Mediante SQL obtener los nombres de las *PlayList* que tienen más de 10 *tracks* de álbumes de "Iron Maiden" ¿Es posible resolver esta consulta usando AR y CRT? Justifique la respuesta.

```
Select      P.name, Count(*)
From Playlist P
  Inner Join PlaylistTrack Plt
    On Plt.PlaylistId = P.PlaylistId
  Inner Join Track T
    On T.TrackId = Plt.TrackId
  Inner Join Album Alb
    On T.AlbumId = Alb.AlbumId
  Inner Join Artist Art
    On Alb.ArtistId = Art.ArtistId
Where Art.name = "Iron Maiden"
Group by P.PlaylistId
Having Count(*) > 10
```

AR:

$\rho(T, \text{Track})$ $\rho(A, \text{Artist})$ $\rho(\text{Alb}, \text{Album})$
 $\rho(P, \text{Playlist})$ $\rho(\text{PLT}, \text{Playlist Track})$

$\rho(\text{TIM}, T(x) \text{ Alb}(x) A)$
 " T.albumId Alb.artistId
 Tracks =
 Iron Maiden Alb.albumId A.artistId

$\rho(\text{PIM}, \text{PLT} \times \text{TIM})$
 ↳ Playlists
 ↳ Iron
 ↳ Maiden
 ↳ PLT.TrackId
 ↳ TIM.TrackId

$$S(PIM1, \dots, PIMn, DIM)$$
$$\rho(P_{I+10}, P_{I_{N1}}) \left(\begin{array}{c} \sigma(P_{I_{N1}}.PlaylistId \\ = \\ \dots \\ = \\ P_{I_{N11}}.PlaylistId \wedge \\ P_{I_{N1}}.TrackId \\ \neq \\ \dots \\ \neq \\ P_{I_{N11}}.TrackId \end{array} \right) (P_{I_{N1}} \times \dots \times P_{I_{N11}})$$
$$\rho(\text{Res}, \pi_{P.\text{Name}} \left(\begin{array}{c} P \text{ l + l o} \\ \text{I d s} \end{array} \right) \times \begin{array}{c} P \\ P \text{ l + l o I d s . P l a y l i s t I d} \\ = \\ P . P l a y l i s t I d \end{array} \right)$$

CRT: también se puede:

- El Álgebra Relacional (AR) es un lenguaje de consulta asociado al Modelo Relacional (MR).
- Una consulta de AR recibe como entrada instancias de relación.
- A su vez, una consulta de AR devuelve también instancias de relación.
- Cualquier consulta escrita en AR puede también expresarse en CRT y viceversa, o sea, ambos lenguajes tienen igual poder expresivo.
- Las consultas de AR devuelven tuplas que no están repetidas.
- Una consulta de AR describe paso a paso (componiendo operadores) cómo se llega a calcular el resultado.

(f) Mediante SQL indicar cuantos álbumes tiene cada *PlayList*. Debe devolver nombre de la *PlayList* y cantidad de álbumes.

```
select P.name, Count(Distinct alb.album_id)
from Playlist P
inner join PlaylistTrack P1t
    on P1t.PlaylistId = P.PlaylistId
inner join Track T
    on P1t.TrackId = T.TrackId
inner join Album Alb
    on Alb.AlbumId = T.AlbumId
inner join Artist A
    on Alb.ArtistId = A.ArtistId
group by P.PlaylistId;
```

(g) Mediante SQL listar los nombres de los empleados (*Employee*) mayores de 25 años que tienen al menos una factura (*Invoice*) con más de 10 ítems.

```
Select e.first_name, e.last_name
From employees e
Inner join Customer c
    on c.supportRepId = e.employee_id
Inner join Invoice i
    on i.CustomerId = c.CustomerId
Inner join InvoiceLine il
    on il.InvoiceId = i.invoiceId
Where (date_part('year', now()) -
       date_part('year', Birth-date)) > 25
Group By e.employee_id, il.InvoiceId
Having Count(il.invoice-Line_id) > 10
```

(h) Mediante SQL resolver el punto b) incluyendo en el listado los clientes que no tienen ninguna factura.

```
Select  C.First Name,  
        C.Last Name,  
From    Customer C  
Left Outer Join Invoice I  
    on   I.CustomerId = C.CustomerId
```

otra forma: *esto trae los clientes sin facturas*

```
Select  C.First Name,  
        C.Last Name,  
From    Customer C  
Where   C.CustomerId Not In  
        (Select i.CustomerId  
         From Invoice
```

(i) Mediante SQL listar los nombres de los empleados que soportan clientes con más de 10 facturas.

```
Select  e.FirstName,  
        e.LastName,  
From    Employee e  
Inner Join Customer c  
        On  c.SupportRepId = e.EmployeeId  
Inner Join Invoice i  
        On  i.CustomerId = c.CustomerId  
Group by e.EmployeeId  
Having c(I.InvoiceId) > 10
```

(j) Mediante SQL listar los empleados junto a su jefe. Las tuplas resultantes tendrán la siguiente forma: (nombre empleado (*FirstName*), apellido de empleado (*LastName*), nombre jefe, apellido de jefe)

```
Select  e1.FirstName,  
        e1.LastName, e2.FirstName AS NombreJefe,  
        e2.LastName AS ApellidoJefe  
From    Employee e1  
Inner Join Employee e2  
        On e1.ReportsTo = e2.EmployeeId
```

(k) Resolver el ítem anterior pero que no falte ningún empleado en el listado

```
SELECT e1.FirstName,  
       e1.LastName, e2.FirstName AS NombreJefe  
FROM Employee e1, e2.LastName AS ApellidoJefe  
LEFT OUTER JOIN Employee e2  
ON e1.ReportsTo = e2.EmployeeId
```

(l) Obtener mediante SQL el promedio de *tracks* comprados en las facturas de cada cliente. Es decir si en una factura compro 8 *tracks* y en otra 4 el promedio es 6.

```
SELECT AVG(suma)  
FROM (SELECT SUM(quantity) AS suma  
      FROM InvoiceLine IL  
      GROUP BY InvoiceId)
```


(m) Obtener para cada empleado el total de *tracks* del genero "Rock" comprados por los clientes que soporta.

```
Select e.FirstName
From employee e
Inner Join Customer C
    On C.SupportRepId = e.EmployeeId
Inner Join Invoice i
    On i.CustomerId = C.CustomerId
Inner Join InvoiceLine IL
    On i.invoiceId = IL.invoiceId
Inner Join Track T
    On IL.trackId = T.TrackId
Inner Join Genre g
    on g.genreId = T.genreId
Where g.name = "Rock"
Group by e.EmployeeId
```